The Journal of Open Source Software

OptimalControl.jl: A Julia package for modeling and solving optimal control problems with ODEs

Jean-Baptiste Caillau [1], **Olivier Cots** [2¶], **Joseph Gergaud** [2], **Pierre Martinon** [3], **and Sophia Sed** [4]

1 Université Côte d'Azur, CNRS, Inria, LJAD, France **2** Université Toulouse, CNRS, ENSEEIHT-IRIT, France **3** CAGE team, Inria Paris, France **4** Inria Sophia Antipolis Méditerranée, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

OptimalControl.jl (Caillau, Cots, Gergaud, Martinon, & Sed, 2024) is a Julia (Bezanson et al., 2017) package for modeling and solving optimal control problems governed by ordinary differential equations (ODEs). As the core of the control-toolbox ecosystem, it provides a unified framework that supports both direct and indirect solution methods with applications spanning aerospace, medical imaging, epidemiology, and quantum control.

The package features an expressive domain-specific language (DSL) built around the `@def` macro, enabling users to define control problems using notation that closely resembles standard mathematical formulations. Problems are solved through direct transcription, converting the continuous problem into a nonlinear program (NLP) using discretization schemes including Euler, trapezoidal, midpoint, and high-order Gauss-Legendre collocation. Alternatively, indirect shooting methods based on Pontryagin's Maximum Principle can be employed. The architecture relies on a modeler-solver separation that provides a modular and extensible foundation, enabling seamless execution on both CPU and GPU with minimal user intervention.

The ecosystem includes extensive tutorial resources, a benchmark problem collection (OptimalControlProblems.jl (Caillau, Cots, Gergaud, & Martinon, 2024) with formulations in OptimalControl DSL and JuMP (Lubin et al., 2023)), and performance comparison tools (CTBenchmarks.jl). Integration with Julia's ecosystem enables access to state-of-the-art tools: NLP solvers IPOPT (Wächter & Biegler, 2006) and MadNLP.jl (Shin et al., 2021, 2024), automatic differentiation and NLP modeling through JuliaSmoothOptimizers' ADNLPModels (Migot et al., 2023), GPU acceleration via ExaModels.jl (Shin et al., 2023), numerical integration from SciML's DifferentialEquations.jl (Rackauckas & Nie, 2017), and visualization through Plots.jl (Christ et al., 2023).

Statement of Need

OptimalControl.jl is, to our knowledge, the only Julia package that unifies both direct and indirect methods for optimal control within a single, coherent framework.

This unified approach addresses a gap in the current optimal control software landscape. Existing tools are fragmented across programming languages and paradigms. Legacy packages such as HamPath (Caillau et al., 2012), NutoPy (Caillau et al., 2022), and COTCOT (Bonnard et al., 2007) implement sophisticated indirect methods but rely on Fortran implementations with MATLAB or Python interfaces, leading to complex and less extensible workflows. Proprietary solvers like GPOPS-II (Patterson & Rao, 2014) limit transparency and reproducibility. Open-source tools such as BOCOP (Bonnans et al., 2017), ACADO (Houska et al., 2011), and

acados (Verschueren et al., 2021) provide valuable direct method implementations but are not natively integrated into a modern, high-level scientific computing ecosystem. Python-based tools such as Dymos (Falck et al., 2021), Pyomo.DAE (Nicholson et al., 2018), do-mpc (Fiedler et al., 2023), and GEKKO (Beal et al., 2018) offer complementary capabilities but are similarly limited to direct transcription approaches within the Python ecosystem.

Within Julia, most existing packages target specific domains: RobustAndOptimalControl.jl for linear systems, QuantumControl.jl and Piccolo.jl for quantum optimal control, DirectTrajectoryOptimization.jl for trajectory problems, LinearMPC.jl (Arnström et al., 2022) and ModelPredictiveControl.jl (Gagnon et al., 2024) for model predictive control, and InfiniteOpt.jl (Pulsipher et al., 2022) for infinite-dimensional optimization. In contrast with InfiniteOpt.jl that is designed as an extension of JuMP.jl, our plan is to provide a modeler agnostic approach, able to accept general Julia code and leverage various optimization solvers. Besides, contributing to general-purpose NLP modeling frameworks like JuMP or CasADi (Andersson et al., 2019) would not address the specific needs of optimal control: these tools lack native support for shooting methods based on Pontryagin's Maximum Principle and that heavily rely on differential geometric primitives.

OptimalControl.jl fills this gap by providing a DSL that matches mathematical notation, multiple discretization schemes and shooting methods, with planned support for homotopy continuation methods. The modeler-solver separation enables complementary use with InfiniteOpt.jl (JuMP-based) through alternative NLP modeling backends (currently ADNLPModels, ExaModels) and solvers. GPU acceleration and an ecosystem with domain-specific applications, tutorials, and benchmarking tools complete the offering. Target users include researchers and engineers working in optimal control, control theorists developing new algorithms, and students learning optimal control through interactive tutorials.

State of the Field

Comparison with existing software

OptimalControl.jl requires Julia version 1.10 or later and is registered in the Julia General registry, enabling straightforward installation via `Pkg.add("OptimalControl")`.

- **Legacy tools (COTCOT, HamPath, NutoPy)**: These Fortran packages excel at indirect methods and homotopy continuation but require multi-language setup (Fortran plus Matlab / Python). OptimalControl.jl provides both direct and indirect methods in pure Julia with straightforward installation via package manager.
- **Direct method tools (BOCOP, ACADO, GPOPS-II, acados, nosnoc)**: Strong direct method implementations: GPOPS-II (Patterson & Rao, 2014) delivers mature methods with MATLAB and C++ implementations but is proprietary; acados (Verschueren et al., 2021) targets real-time MPC on embedded systems; nosnoc (Nurkanović & Diehl, 2022) specializes in nonsmooth optimal control; CasADi (Andersson et al., 2019), used as symbolic backend by several of these tools, is a general NLP modeler rather than an optimal control solver. OptimalControl.jl offers an open-source alternative with expressive DSL, native Julia ecosystem integration, GPU support, and unified direct and indirect approaches.
- **Julia packages**: RobustAndOptimalControl.jl targets linear systems; QuantumControl.jl, Piccolo.jl and DirectTrajectoryOptimization.jl serve specific domains; LinearMPC.jl (Arnström et al., 2022) and ModelPredictiveControl.jl (Gagnon et al., 2024) focus on model predictive control. InfiniteOpt.jl addresses a very rich range of problems, including optimization on PDE's or with chance constraints, focusing on direct transcription methods. OptimalControl.jl works seamlessly on both CPU and GPU, and adds tools to do shooting in a unified framework plus systematic benchmarking through OptimalControlProblems.jl and CTBenchmarks.jl.

DRAFT

The Journal of Open Source Software

Illustrative Example

The following example illustrates both direct and indirect solution approaches for a constrained energy minimization problem. The workflow demonstrates a practical strategy: a direct method on a coarse grid first identifies the problem structure and provides an initial guess for the indirect method, which then computes a solution up to arbitrary precision via shooting based on Pontryagin's Maximum Principle.

Definition of the optimal control problem:

```
# Problem definition: energy-optimal control with state constraint
using OptimalControl
t0 = 0
tf = 1
x0 = [-1, 0]
x_target = [0, 0]
v_max = 1.2
ocp = @def begin
    t ∈ [t0, tf], time
    x = (q, v) ∈ R², state
    u ∈ R, control
    x(t0) == x0              # initial condition
    x(tf) == x_target        # terminal condition
    v(t) ≤ v_max             # state constraint
    ∂(x)(t) == [v(t), u(t)]  # dynamics
    0.5∫(u(t)^2) → min       # minimize control energy
end
```

Direct method: The problem is transcribed to an NLP using a midpoint discretization scheme and solved with an interior-point solver.

```
using NLPModelsIpopt
direct_sol = solve(ocp; grid_size=50)
```

Extract initial guess from direct solution: The initial costate and the two switching times are recovered from the direct solution to initialize the indirect method.

```
t = time_grid(direct_sol)           # the time grid as a vector
x = state(direct_sol)               # the state as a function of time
p = costate(direct_sol)             # the costate as a function of time
p0 = p(t0)                          # initial costate
g(x) = v_max - x[2]                 # constraint: g(x) ≥ 0
I = findall(t -> g(x(t)) ≤ 1e-3, t) # times where constraint is active
t1 = t[first(I)]                    # entry time
t2 = t[last(I)]                     # exit time
initial_guess = [p0..., t1, t2]     # initial guess for shooting
```

Indirect method: The indirect method applies Pontryagin's Maximum Principle. The solution has three phases (unconstrained-constrained-unconstrained arcs), requiring definition of Hamiltonian flows for each phase and a shooting function to enforce boundary conditions.

```
using OrdinaryDiffEq, NonlinearSolve

# Define Hamiltonian flows based on Pontryagin's Maximum Principle
f_interior = Flow(ocp, (x, p) -> p[2]) # u(x, p) is p[2]
f_boundary = Flow(ocp, (x, p) -> 0,    # u(x, p) is 0
                       (x, u) -> g(x), # when g(x) = 0 (constrained arc)
                       (x, p) -> p[1]) # associated multiplier is p[1]
```

DRAFT

The Journal of Open Source Software

```julia
# Shooting function
function shoot!(s, ξ, _) # unused last parameter (required by NonlinearSolve)
    p0, t1, t2 = ξ[1:2], ξ[3], ξ[4]
    x1, p1 = f_interior(t0, x0, p0, t1)    # flow on unconstrained arc [t0, t1]
    x2, p2 = f_boundary(t1, x1, p1, t2)    # flow on constrained arc [t1, t2]
    xf, pf = f_interior(t2, x2, p2, tf)    # flow on unconstrained arc [t2, tf]
    s[1:2] = xf - x_target                 # terminal condition
    s[3] = g(x1)                           # entry condition on constraint
    s[4] = p1[2]                           # switching phase condition
end

# Solve shooting problem
shooting_sol = solve(NonlinearProblem(shoot!, initial_guess))
p0, t1, t2 = shooting_sol.u[1:2], shooting_sol.u[3], shooting_sol.u[4]
```

Trajectory reconstruction and visualization: The trajectory is reconstructed by concatenating the three flows using the `*` operator (and indicating the internal times when to concatenate, `t1` and `t2`). Then both solutions are compared visually.

```julia
# Reconstruct trajectory by concatenating flows
φ = f_interior * (t1, f_boundary) * (t2, f_interior)
indirect_sol = φ((t0, tf), x0, p0; saveat=range(t0, tf, 100))

# Compare both solutions
using Plots
plot(direct_sol; label="Direct")
plot!(indirect_sol; label="Indirect", color=2, linestyle=:dash)
```

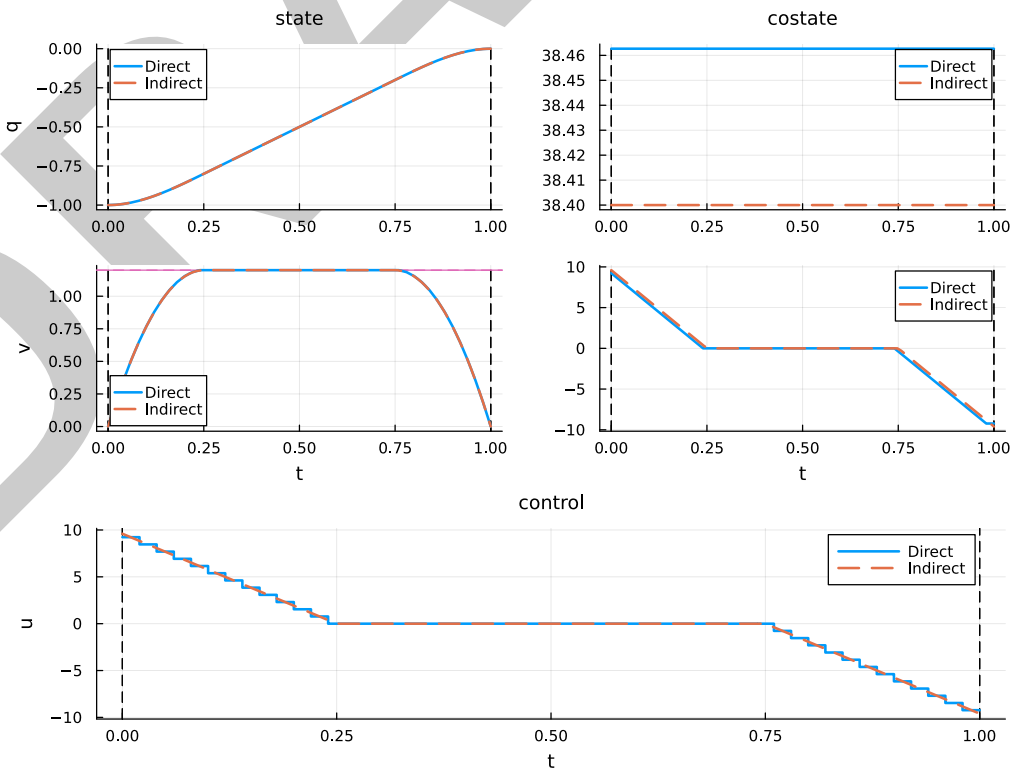

Figure 1: Comparison of direct and indirect solutions showing state trajectories (position and velocity), costate variables, and control inputs. The three arc structure is very well captured by the first direct solve, allowing convergence to arbitrary precision of the final shooting.

DRAFT

Software Design

The package architecture balances expressiveness, performance, and extensibility through modular design. The core is organized across internal packages within the control-toolbox organization, each hosted as an independent Julia package and registered in the General registry: CTBase.jl defines the core types (optimal control models, solutions, initial guesses) and their accessors; CTParser.jl implements the `@def` DSL macro; CTDirect.jl handles direct transcription and NLP interfacing; and CTFlows.jl provides Hamiltonian flows and shooting for indirect methods. OptimalControl.jl re-exports these packages as a unified entry point. Contributors interested in a specific functionality can work directly on the relevant sub-package, each of which has its own documentation, tests, and continuous integration. The delegation of specific responsibilities is as follows:

- **DSL parsing**: MLStyle.jl (Thautwarm & contributors, 2023) enables pattern matching on abstract syntax trees generated from `@def` macro invocations. This design choice separates problem specification from solution methods, and allows a syntax as close as possible to the mathematical description of the problem.
- **Problem models and solutions**: Structured types represent optimal control models, solutions, and initial guesses. Each type provides textual visualization, accessor methods for introspection, and visual plotting capabilities for solutions. Extensible abstractions enable addition of new problem classes without modifying core code.
- **Discretization**: Direct transcription converts continuous problems into NLPs. Supporting multiple discretization schemes (Euler, midpoint, trapezoidal, Gauss-Legendre) required careful abstraction to share transcription logic while allowing scheme-specific implementations.
- **Modelers and solvers**: We chose a clear modeler-solver separation: discretization produces ADNLPModels (able to deal with arbitrary user defined functions) or ExaModels (Shin et al., 2023) instances compatible with multiple NLP solvers (IPOPT via NLPModelsIpopt.jl (Orban et al., 2023), Knitro (Byrd et al., 2006), MadNLP, and Uno (Vanaret & Leyffer, 2026)). These NLP solvers rely on linear solvers such as MUMPS.jl (Amestoy et al., 2001, 2019; Montoison et al., 2024) for CPU computations or CUDSS.jl (NVIDIA Corporation, 2025) for GPU acceleration with MadNLP. Automatic differentiation via ForwardDiff.jl (Revels et al., 2016) and DifferentiationInterface.jl (Dalle & Hill, 2026) avoids manual derivative coding.
- **Indirect methods**: Hamiltonian flows integrate with DifferentialEquations.jl to access adaptive stepping, event handling, and multiple ODE solvers without reimplementation. Shooting methods rely on NonlinearSolve.jl (Pal et al., 2025) or MINPACK.jl for root-finding. Future extensions will incorporate homotopy continuation methods leveraging bifurcation analysis tools like BifurcationKit.jl (Veltz, 2020).

Key design trade-offs:

1. **DSL vs. programmatic API**: We prioritized a mathematical DSL to reduce cognitive load for domain experts, accepting increased parsing complexity handled internally. Currently we do not address optimization of PDE or stochastic systems.
2. **GPU acceleration strategy**: The modeler-solver separation enables seamless CPU-to-GPU transition. Users simply select the appropriate modeler-solver pair: ExaModels with MadNLP and the CUDSS linear solver for GPU execution. This modular approach minimizes maintenance burden while enabling GPU performance without reimplementing transcription logic. As for now, GPU support is limited to the ExaModels + MadNLP modeler-solver combination.
3. **Method coverage**: Supporting both direct and indirect approaches increases code complexity but serves distinct user needs: direct methods for constrained problems with

DRAFT

many variables, indirect methods for theoretical analysis and smaller problems requiring high accuracy. Both approaches resort to iterative solvers and may converge or not, *e.g.* depending on the initial guess. In the case of optimization solvers, the full output status of the solver is returned allowing *a posteriori* analysis.

Research Impact Statement

OptimalControl.jl has been applied in published research across multiple domains. Independent external adoption demonstrates the package's broader impact: Ferede et al. (2025) used it for drone racing trajectory optimization, Caio & Silva (2025) for epidemiological models using indirect methods, Morsky (2025) for vaccination strategies under social norms, Opmeer (2025) for optimal harvesting of age-structured populations, and Evangelakos et al. (2025) for fast charging protocols for quantum batteries.

Research by close collaborators further demonstrates the package's versatility across aerospace (Herasimenka et al. (2026)), epidemiology (Bliman et al. (2026)), quantum control (Beschastnyi et al. (2025)), microbial growth control (Innerarity Imizcoz et al. (2025)), and theoretical optimal control (Bouali et al. (2025); Lutz & Privat (2025); Bayen et al. (2026); Bonnard et al. (2026)).

The control-toolbox organization hosts domain-specific application packages built on OptimalControl.jl: medical imaging optimization (MagneticResonanceImaging.jl), gene regulatory networks (PWLdynamics.jl), spacecraft orbital transfers (Kepler.jl), epidemiological modeling (SIRcontrol.jl), and variational calculus (CalculusOfVariations.jl).

The package serves educational purposes through Tutorials.jl covering topics from linear-quadratic regulators to Model Predictive Control. These resources are used in academic courses and workshops. OptimalControlProblems.jl provides standardized test problems formulated in both OptimalControl DSL and JuMP, enabling systematic performance comparisons through CTBenchmarks.jl.

Development involves international collaborations with CNES (French space agency), Thales Alenia Space, Inria, and CNRS. The package development began in September 2022, with the first public release in February 2023. As of March 2026, OptimalControl.jl has published more than 40 releases with contributions from multiple developers across the control-toolbox ecosystem packages, demonstrating sustained community engagement.

AI Usage Disclosure

The core software implementation, algorithms, and architectural design of OptimalControl.jl were developed by the authors. Generative AI has been used to assist with code refactoring and generation of unit and integration tests. This paper was drafted by the authors and subsequently revised with AI assistance for restructuring according to JOSS format requirements and language editing. All technical content, examples, and claims reflect the authors' work and judgment.

Acknowledgements

We thank the control-toolbox community and the broader Julia ecosystem for contributions that shaped the package design and educational resources. Development has been supported by partnerships with CNES, Thales Alenia Space, Inria, and CNRS. The software is distributed under the MIT license (see LICENSE) and uses centralized GitHub Actions workflows for continuous integration, documentation, and testing across the ecosystem. We welcome community contributions following the guidelines in CONTRIBUTING.md.

References

Amestoy, P. R., Buttari, A., L'Excellent, J.-Y., & Mary, T. (2019). Performance and Scalability of the Block Low-Rank Multifrontal Factorization on Multicore Architectures. *ACM Transactions on Mathematical Software*, *45*(1), 2:1–2:26. https://doi.org/10.1145/3242094

Amestoy, P. R., Duff, I. S., Koster, J., & L'Excellent, J.-Y. (2001). A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, *23*(1), 15–41.

Andersson, J. A., Gillis, J., Horn, G., Rawlings, J. B., & Diehl, M. (2019). CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, *11*(1), 1–36. https://doi.org/10.1007/s12532-018-0139-4

Arnström, D., Bemporad, A., & Axehill, D. (2022). A dual active-set solver for embedded quadratic programming using recursive LDL^T updates. *IEEE Transactions on Automatic Control*, *67*(8), 4362–4369. https://doi.org/10.1109/TAC.2022.3176430

Bayen, T., Bouali, A., & Bourdin, L. (2026). Minimum time problem for the double integrator with a loss-of-control region. *Nonlinear Analysis: Hybrid Systems*, *60*, 101681. https://doi.org/10.1016/j.nahs.2026.101681

Beal, L. D. R., Hill, D. C., Martin, R. A., & Hedengren, J. D. (2018). GEKKO optimization suite. *Processes*, *6*(8), 106. https://doi.org/10.3390/pr6080106

Beschastnyi, I., Dell'Elce, L., Pomet, J.-B., Sacchelli, L., & Tinoco, D. (2025). Pulse control of affine systems with applications to quantum control. *2025 IEEE 64th Conference on Decision and Control (CDC)*, 3769–3774. https://doi.org/10.1109/CDC57313.2025.11312971

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Bliman, P.-A., Bouali, A., Loisel, P., Rapaport, A., & Virelizier, A. (2026). On the problem of minimizing the epidemic final size for SIR model by social distancing. *Mathematical Biosciences and Engineering*, *23*(3), 567–593. https://doi.org/10.3934/mbe.2026022

Bonnans, J. F., Martinon, P., Giorgi, D., Grélard, V., Maindrault, S., & Tissot, O. (2017). The bocop and BocHop optimization toolboxes. *IFAC-PapersOnLine*, *50*(1), 8849–8854. https://doi.org/10.1016/j.ifacol.2017.08.1745

Bonnard, B., Caillau, J.-B., & Trélat, E. (2007). Second order optimality conditions in the smooth case and applications in optimal control. *ESAIM: Control, Optimisation and Calculus of Variations*, *13*(2), 207–236. https://doi.org/10.1051/cocv:2007012

Bonnard, B., Cots, O., & Privat, Y. (2026). The zermelo navigation problem on the 2-sphere of revolution: An optimal control perspective with applications to micromagnetism. *Nonlinearity*, *39*(1), 015006. https://doi.org/10.1088/1361-6544/ae2cd9

Bouali, A., Rapaport, A., & Bayen, T. (2025). *Regularization of optimal control problems on stratified domains using additional controls*. https://hal.inrae.fr/hal-04928858

Byrd, R. H., Nocedal, J., & Waltz, R. A. (2006). Knitro: An integrated package for nonlinear optimization. In G. Di Pillo & M. Roma (Eds.), *Large-scale nonlinear optimization* (pp. 35–59). Springer US. https://doi.org/10.1007/0-387-30065-1_4

Caillau, J.-B., Cots, O., & Gergaud, J. (2012). Differential continuation for regular optimal control problems. *Optimization Methods and Software*, *27*(2), 177–196. https://doi.org/10.1080/10556788.2011.593625

Caillau, J.-B., Cots, O., Gergaud, J., & Martinon, P. (2024). *OptimalControlProblems.jl: a collection of optimal control problems with ODEs in Julia*. https://doi.org/10.5281/zenodo.17013180

DRAFT

Caillau, J.-B., Cots, O., Gergaud, J., Martinon, P., & Sed, S. (2024). *OptimalControl.jl: a Julia package to model and solve optimal control problems with ODEs*. https://doi.org/10.5281/zenodo.13336563

Caillau, J.-B., Cots, O., & Martinon, P. (2022). Ct: Control toolbox – numerical tools and examples in optimal control. *IFAC-PapersOnLine*, *55*(16), 13–18. https://doi.org/10.1016/j.ifacol.2022.08.074

Caio, P., & Silva, C. J. (2025). Application of indirect methods to optimal control problems in epidemiology. *CONTROLO 2024*, 444–454. https://doi.org/10.1007/978-3-031-81724-3_40

Christ, S., Schwabeneder, D., Rackauckas, C., Borregaard, M. K., & Breloff, T. (2023). *Plots.jl – a user extendable plotting API for the julia programming language*. https://doi.org/10.5334/jors.431

Dalle, G., & Hill, A. (2026). A common interface for automatic differentiation. *Journal of Machine Learning Research*, *27*(25), 1–13. http://jmlr.org/papers/v27/25-1024.html

Evangelakos, V., Paspalakis, E., & Stefanatos, D. (2025). Fast protocols for charging a three-spin-chain quantum battery. *Scientific Reports*, *15*(1), 45626. https://doi.org/10.1038/s41598-025-30354-1

Falck, R. D., Gray, J. S., Ponnapalli, K., & Wright, T. (2021). Dymos: A python package for optimal control of multidisciplinary systems. *Journal of Open Source Software*, *6*(59), 2809. https://doi.org/10.21105/joss.02809

Ferede, R., Blaha, T., Lucassen, E., De Wagter, C., & Croon, G. C. H. E. de. (2025). One net to rule them all: Domain randomization in quadcopter racing across different platforms. *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 6357–6363. https://doi.org/10.1109/ICRA55743.2025.11128790

Fiedler, F., Karg, B., Lüken, L., Brandner, D., Heinlein, M., Brabender, F., & Lucia, S. (2023). Do-mpc: Towards FAIR nonlinear and robust model predictive control. *Control Engineering Practice*, *140*, 105676. https://doi.org/10.1016/j.conengprac.2023.105676

Gagnon, F., Thivierge, A., Desbiens, A., & Bagge Carlson, F. (2024). *ModelPredictiveControl.jl: advanced process control made easy in Julia*. https://doi.org/10.48550/arXiv.2411.09764

Herasimenka, A., Baresi, N., Green, L., Morgan, H., Underwood, C. I., Bridges, C., Jason, S., Lucca Fabris, A., & Ryden, K. (2026). Low-thrust trajectory optimization for the moon-enabled sun occultation mission concept. *AIAA SCITECH 2026 Forum*.

Houska, B., Ferreau, H. J., & Diehl, M. (2011). ACADO toolkit—an open-source framework for automatic control and dynamic optimization. *Optimal Control Applications and Methods*, *32*(3), 298–312. https://doi.org/10.1002/oca.939

Innerarity Imizcoz, J., Yabo, A. G., Djema, W., Francis, F., & Gouzé, J.-L. (2025). *Optimal allocation control in microbial growth under a heat-shock*. https://inria.hal.science/hal-05369609

Lubin, M., Dowson, O., Dias Garcia, J., Huchette, J., Legat, B., & Vielma, J. P. (2023). JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*. https://doi.org/10.1007/s12532-023-00239-3

Lutz, K., & Privat, Y. (2025). *Minimal time control of underdamped parametric oscillators*. https://hal.science/hal-05047678

Migot, T., Montoison, A., Orban, D., Soares Siqueira, A., & contributors. (2023). *ADNLPModels.jl: Automatic Differentiation models implementing the NLPModels API*. https://github.com/JuliaSmoothOptimizers/ADNLPModels.jl

Montoison, A., Orban, D., Sweeney, W. R., & contributors. (2024). *MUMPS.jl: A*

DRAFT

Julia Interface to MUMPS (Version 1.4.1). https://github.com/JuliaSmoothOptimizers/MUMPS.jl

Morsky, B. (2025). Vaccination and collective action under social norms. *Bulletin of Mathematical Biology*, *87*(5), 55. https://doi.org/10.1007/s11538-025-01436-y

Nicholson, B., Siirola, J. D., Watson, J.-P., Zavala, V. M., & Biegler, L. T. (2018). Pyomo.dae: A modeling and automatic discretization framework for optimization with differential and algebraic equations. *Mathematical Programming Computation*, *10*(2), 187–223. https://doi.org/10.1007/s12532-017-0127-0

Nurkanović, A., & Diehl, M. (2022). Nosnoc: A software package for numerical optimal control of nonsmooth systems. *IEEE Control Systems Letters*, *6*, 3110–3115.

NVIDIA Corporation. (2025). *NVIDIA cuDSS (Preview): A high-performance CUDA Library for Direct Sparse Solvers*. https://docs.nvidia.com/cuda/cudss/index.html.

Opmeer, M. (2025). Optimal harvesting of a continuously age-structured population with density dependence. *PLOS ONE*, *20*(9), 1–12. https://doi.org/10.1371/journal.pone.0333087

Orban, D., Soares Siqueira, A., & contributors. (2023). *NLPModelsIpopt.jl: A thin IPOPT wrapper for NLPModels* (Version 0.10.1). https://github.com/JuliaSmoothOptimizers/NLPModelsIpopt.jl

Pal, A., Holtorf, F., Larsson, A., Loman, T., Utkarsh, Schäfer, F., Qu, Q., Edelman, A., & Rackauckas, C. (2025). NonlinearSolve.jl: High-performance and robust solvers for systems of nonlinear equations in julia. *ACM Trans. Math. Softw.* https://doi.org/10.1145/3779117

Patterson, M. A., & Rao, A. V. (2014). GPOPS-II: A MATLAB software for solving multiple-phase optimal control problems using hp-adaptive gaussian quadrature collocation methods and sparse nonlinear programming. *ACM Transactions on Mathematical Software*, *41*(1), 1–37. https://doi.org/10.1145/2558904

Pulsipher, J. L., Zhang, W., Hongisto, T. J., & Zavala, V. M. (2022). A unifying modeling abstraction for infinite-dimensional optimization. *Computers & Chemical Engineering*, *156*. https://doi.org/10.1016/j.compchemeng.2021.107567

Rackauckas, C., & Nie, Q. (2017). Differentialequations.jl–a performant and feature-rich ecosystem for solving differential equations in julia. *Journal of Open Research Software*, *5*(1), 15.

Revels, J., Lubin, M., & Papamarkou, T. (2016). *Forward-mode automatic differentiation in Julia*. https://arxiv.org/abs/1607.07892

Shin, S., Anitescu, M., & Pacaud, F. (2024). Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, *236*, 110651. https://doi.org/10.1016/j.epsr.2024.110651

Shin, S., Coffrin, C., Sundar, K., & Zavala, V. M. (2021). Graph-based modeling and decomposition of energy infrastructures. *IFAC-PapersOnLine*, *54*(3), 693–698. https://doi.org/10.1016/j.ifacol.2021.08.322

Shin, S., Pacaud, F., & Anitescu, M. (2023). *Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods*. https://arxiv.org/abs/2307.16830

Thautwarm, & contributors. (2023). *MLStyle.jl: Functional programming and pattern matching for Julia*. https://thautwarm.github.io/MLStyle.jl

Vanaret, C., & Leyffer, S. (2026). *Implementing a unified solver for nonlinearly constrained optimization*.

DRAFT

Veltz, R. (2020). *BifurcationKit.jl*. Inria Sophia-Antipolis. https://hal.archives-ouvertes.fr/hal-02902346

Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., Duijkeren, N. van, Zanelli, A., Novoselnik, B., Albin, T., Quirynen, R., & Diehl, M. (2021). Acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*. https://doi.org/10.1007/s12532-021-00208-8

Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, *106*(1), 25–57. https://doi.org/10.1007/s10107-004-0559-y

DRAFT